

CH6 Kafka

Dr. Dalel Ayed Lakhal

Qu'est-ce qu'un système de messaging?

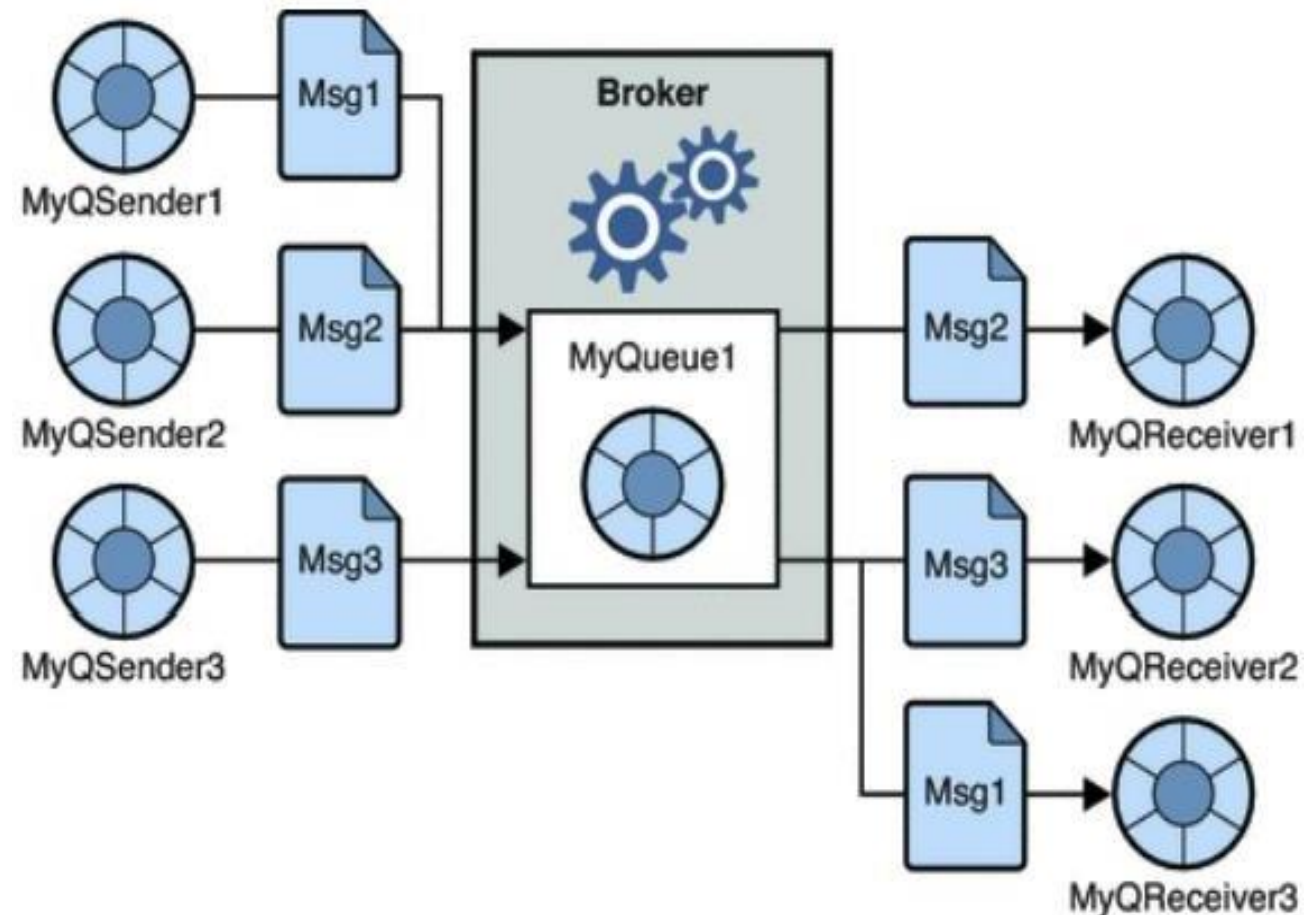
- Qu'est-ce qu'un message ?
 - N'importe quelle suite d'octets (byte array).
 - Généralement au format clé/valeur.
 - On essaie de structurer la valeur avec un format simple (JSON, CSV, XML) ou une bibliothèque plus avancée.
 - Les données doivent être transférées, donc elles sont sérialisées/désérialisées.
 - Les messages sont indexés avec un compteur appelé offset pour chaque partition

Un système de messaging (Messaging System) est responsable du transfert de données d'une application à une autre, de manière à ce que les applications puissent se concentrer sur les données sans s'inquiéter de la manière de les partager ou de les collecter

- Deux types de patrons de messaging existent: Les systèmes "point à point" et les systèmes "publish-subscribe".

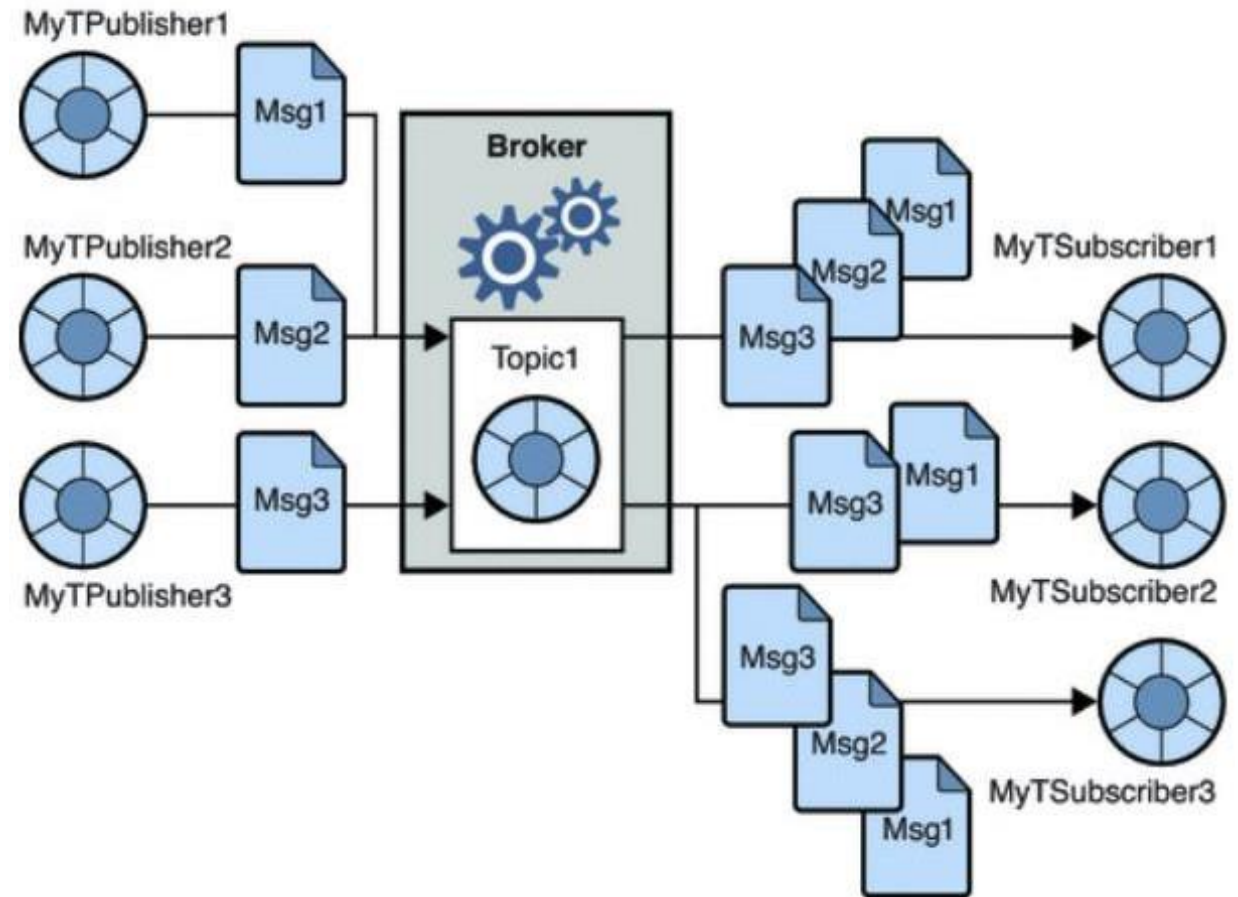
1. SYSTÈMES DE MESSAGING POINT À POINT

Dans un système point à point, les messages sont stockés dans une file. un ou plusieurs consommateurs peuvent consommer les messages dans la file, mais un message ne peut être consommé que par un seul consommateur à la fois. Une fois le consommateur lit le message, ce dernier disparaît de la file



2. SYSTÈMES DE MESSAGING PUBLISH/SUBSCRIBE

Dans un système publish-subscribe, les messages sont stockés dans un "topic". Contrairement à un système point à point, les consommateurs peuvent souscrire à un ou plusieurs topics et consommer tous les messages de ce topic.



Producer/Consumer

- Aussi connu comme publish/subscribe
- Deux types d'acteurs:
 - Les publishers/producers: Ils peuvent écrire des messages dans des topics
 - Les subscribers/consumers: Ils sont notifiés quand il y a des nouveaux messages et peuvent les lire

C'est quoi Apache Kafka ?

- Une plateforme distribuée pour les flux d'événements
- Basé sur publish-subscribe

Applications:

- Suivi des activités : Enregistrer les activités des utilisateurs sur un site web. Ces activités sont ensuite utilisées par d'autres applications pour générer des rapports, alimenter des modèles de machine learning, mettre à jour les résultats de recherche, etc.
- Messagerie : Envoyer des e-mails de manière asynchrone.
- Métriques/Journalisation : Les applications publient des métriques qui sont ensuite consommées par un système de surveillance.
- Journal de commit : Publier les modifications d'une base de données sur Kafka. Ces modifications peuvent ensuite être enregistrées dans une base de données, répliquées, ou traitées pour d'autres applications, etc

1. Elle vous permet de publier et souscrire à un flux d'enregistrements.
Elle ressemble ainsi à une file de message ou un système de messaging d'entreprise.
2. Elle permet de stocker des flux d'enregistrements d'une façon tolérante aux pannes.
3. Elle vous permet de traiter (au besoin) les enregistrements au fur et à mesure qu'ils arrivent.

Avantages

1. La fiabilité: Kafka est distribué, partitionné, répliqué et tolérant aux fautes.
2. La scalabilité: Kafka se met à l'échelle facilement et sans temps d'arrêt.
3. La durabilité: Kafka utilise un commit log distribué, ce qui permet de stocker les messages sur le disque le plus vite possible.
- 4. La performance: Kafka a un débit élevé pour la publication et l'abonnement.

A une faible latence et un débit élevé : les messages sont traités rapidement.

- Est tolérant aux pannes.
- Permet de créer un pipeline de données en temps réel.

Inconvénients

- Format des messages fixe : difficile de modifier les messages.
- La rétention est coûteuse : il faut utiliser un autre système pour le stockage à long terme (HDFS).
- Les consommateurs doivent lire l'intégralité des messages : ils ne peuvent pas récupérer un champ spécifique.

Topic Kafka

- Un flux de messages appartenant à une catégorie particulière sur laquelle des producers peuvent écrire et des consumers lire. Les données sont stockées dans des topics.

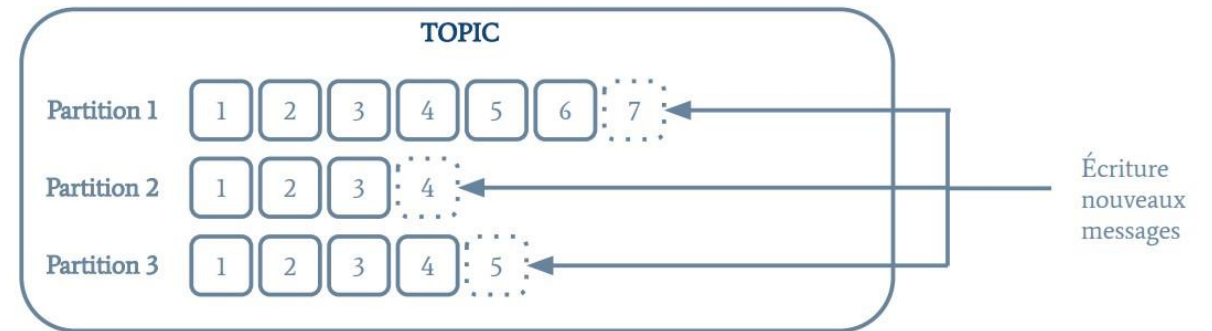


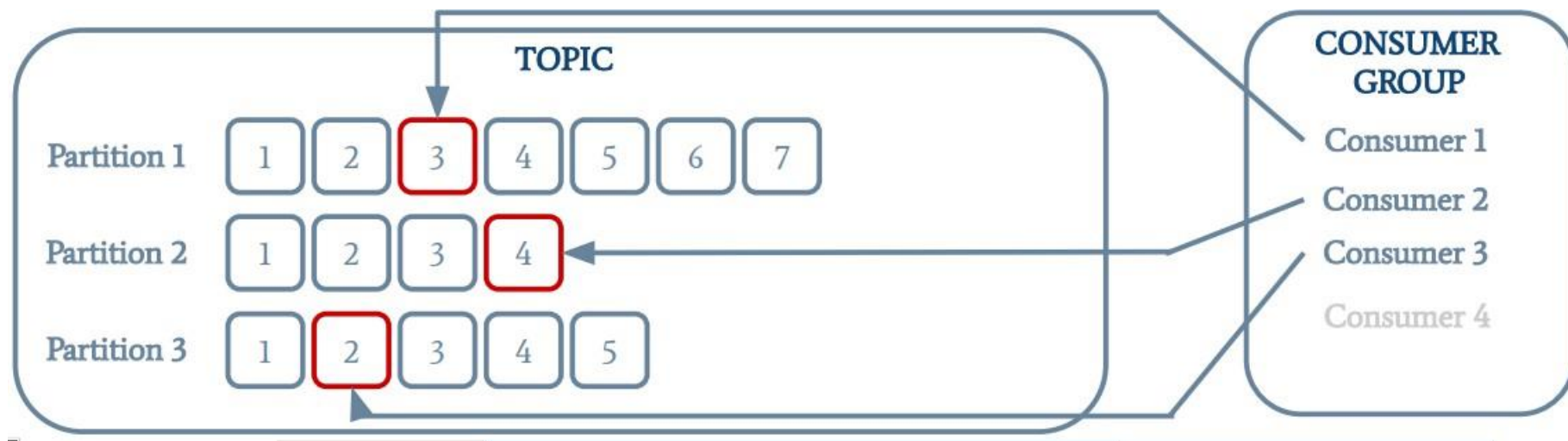
Architecture de Kafka

Partitions

Chaque topic est divisé en partitions. Pour chaque topic, Kafka conserve un minimum d'une partition. Chaque partition contient des messages dans une séquence ordonnée immuable. Une partition est implémentée comme un ensemble de segments de tailles égales.

- Les messages sont stockés dans un topic sous forme de partitions en ajout seulement (append-only).
- Cela permet le traitement groupé des messages, la réplication, la tolérance aux pannes, l'écriture parallèle, un débit élevé, etc.
- La partition d'un message peut être choisie automatiquement par Kafka ou définie via une fonction basée sur une clé





Architecture de Kafka

Offset:

Les enregistrements d'une partition ont chacun un identifiant séquentiel appelé offset, qui permet de l'identifier de manière unique dans la partition.

Répliques:

Les répliques sont des backups d'une partition. Elles ne sont jamais lues ni modifiées par les acteurs externes, elles servent uniquement à prévenir la perte de données

Architecture de Kafka

Producers

- Les producteurs écrivent des messages dans un topic.

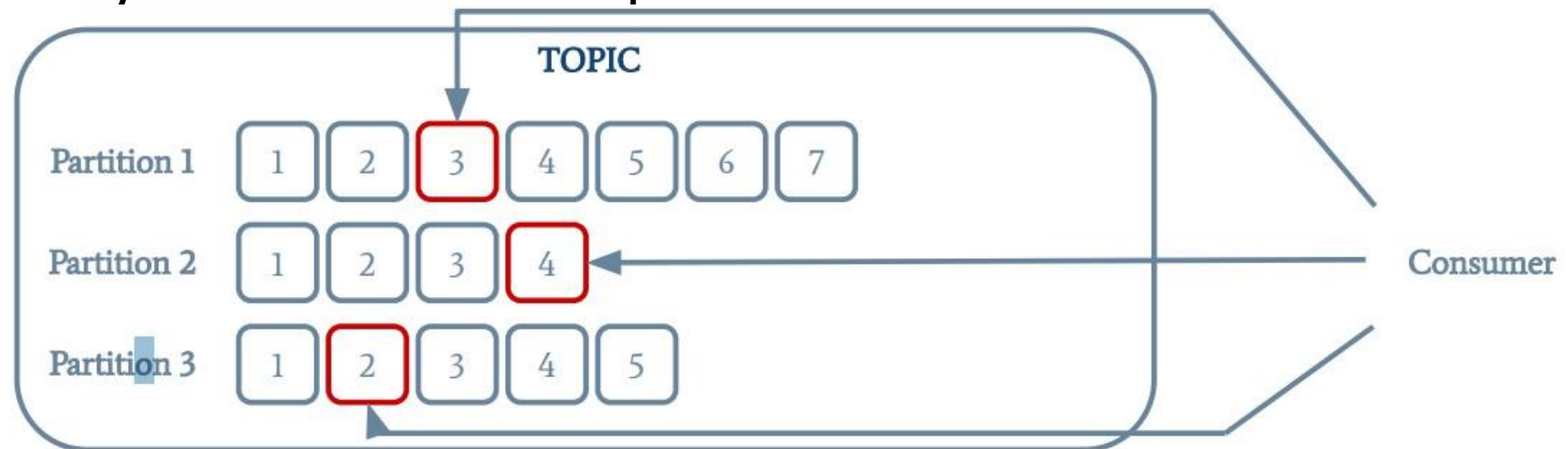
Ils envoient des données aux courtiers Kafka. Chaque fois qu'un producteur publie un message à un courtier, ce dernier rattache le message au dernier segment, ajouté ainsi à une partition. Un producteur peut également envoyer un message à une partition particulière

Les producteurs sont les éditeurs de messages à un ou plusieurs topics Kafka.

Ils envoient des données aux courtiers Kafka. Chaque fois qu'un producteur publie un message à un courtier, ce dernier rattache le message au dernier segment, ajouté ainsi à une partition. Un producteur peut également envoyer un message à une partition particulière

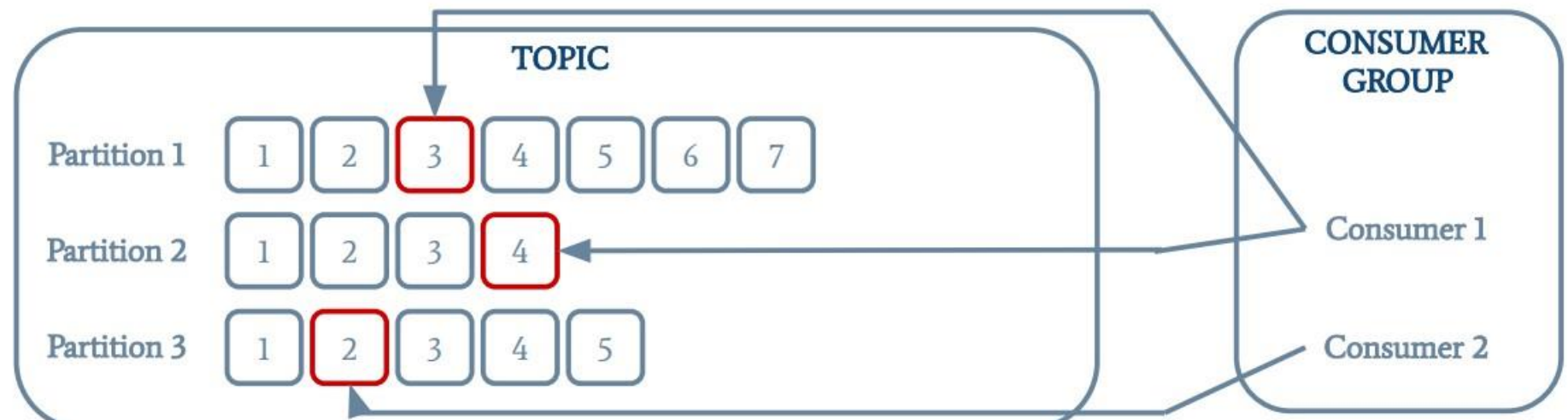
Consumers

- Les consommateurs s'abonnent à un topic donné et lisent les messages.
- Un consommateur garde une trace de son offset actuel pour chaque partition. Il peut ainsi reprendre le traitement plus tard en cas de crash.
- Les consommateurs lisent les données à partir des brokers. Ils souscrivent à un ou plusieurs topics, et consomment les messages publiés en extrayant les données à partir des brokers.



Consumer Groups

- Un groupe de consommateurs est composé de plusieurs consommateurs qui partagent les mêmes offsets.
- Une partition est toujours attribuée à un seul consommateur.
- S'il y a plus de consommateurs que de partitions, les consommateurs supplémentaires attendent.



Architecture de Kafka

6. Cluster:

Un système Kafka ayant plus qu'un seul Broker est appelé cluster Kafka. L'ajout de nouveau brokers est fait de manière transparente sans temps d'arrêt.

9. Leaders:

Le leader est le noeud responsable de toutes les lectures et écritures d'une partition donnée. Chaque partition a un serveur jouant le rôle de leader.

10. Follower:

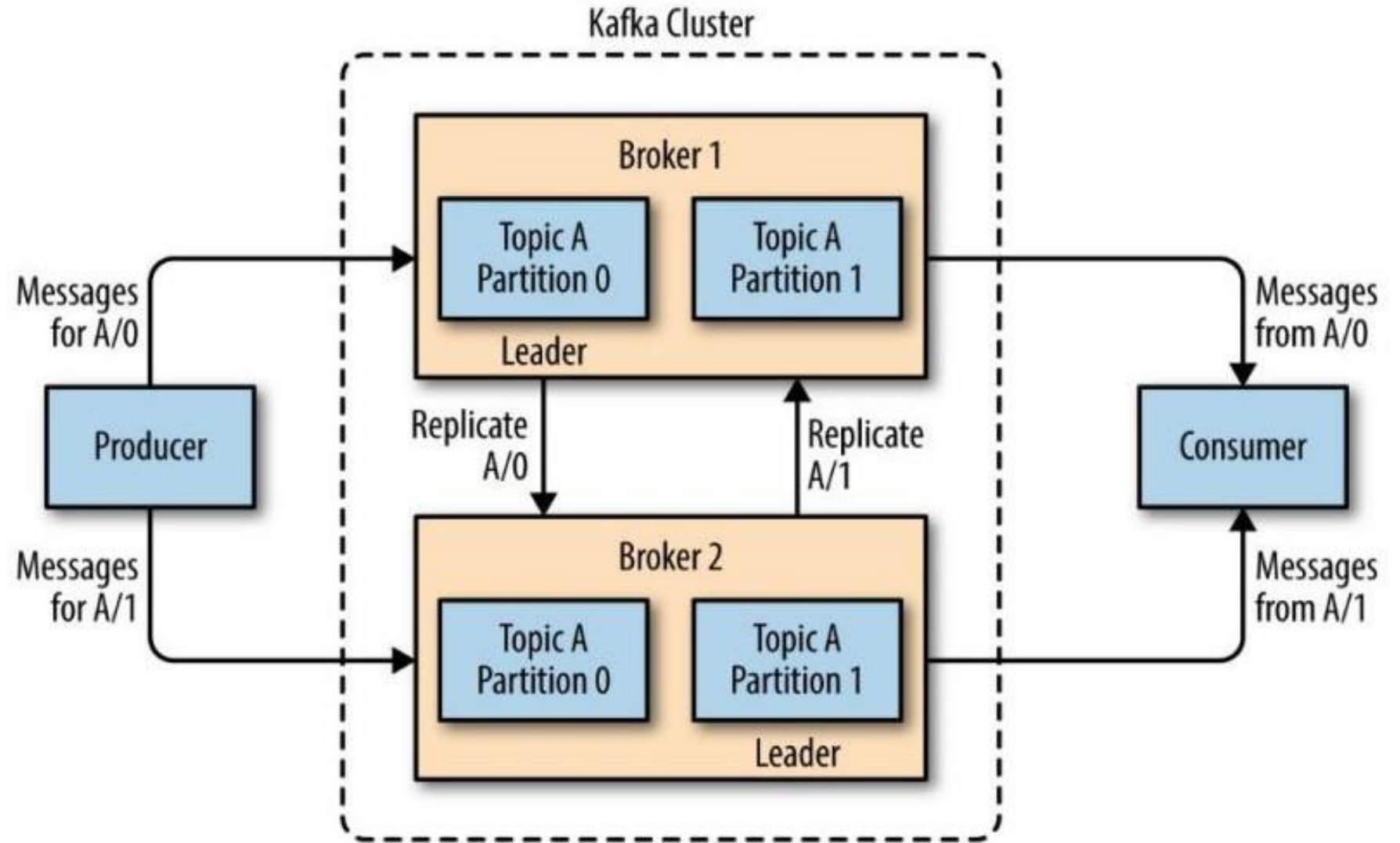
C'est un noeud qui suit les instructions du leader. Si le leader tombe en panne, l'un des followers deviendra automatiquement le nouveau leader.

Brokers

- Un serveur Kafka unique est appelé un broker.
 - Un broker est chargé de répondre aux requêtes des producteurs et des consommateurs et de gérer ses partitions.
 - Un cluster est composé de plusieurs brokers.
 - L'un d'eux est le contrôleur, responsable des tâches administratives comme l'attribution des partitions.
 - Une partition appartient à un seul broker (le leader), mais elle peut être répliquée sur plusieurs brokers

Les brokers (ou courtiers) sont de simples systèmes responsables de maintenir les données publiées. Chaque courtier peut avoir zéro ou plusieurs partitions par topic. Si un topic admet N partitions et N courtiers, chaque courtier va avoir une seule partition. Si le nombre de courtiers est plus grand que celui des partitions, certains n'auront aucune partition de ce topic.

Brokers



Conclusion

- Kafka est un système pour faire passer des messages entre des producteurs et des consommateurs
- Les consommateurs peuvent être agrégés en groupes
 - Le traitement est alors distribué dans le groupe